

Universidad Politecnica Salesiana
Maestria en Ingenieria de Software
Desarrollo de Aplicaciones Empresariales

Sistema de Reserva de Canchas Deportivas

Informe academico final

Frontend 1: Enzo Aliatis
Frontend 2: Jordan Murillo
Backend 1: Galo Suarez
Backend 2: Mauro Cadme

Agosto 2026

Índice

1. Resumen ejecutivo	1
2. Introduccion	1
3. Objetivos	1
3.1. Objetivo general	1
3.2. Objetivos especificos	1
4. Alcance funcional	2
4.1. Detalle de funcionalidades clave	2
5. Roles, permisos y modulos	3
5.1. Roles	3
5.2. Modulos funcionales	3
6. Reglas de negocio	5
7. Arquitectura implementada	6
8. Microfrontends	6
9. Microservicios	10
9.1. Endpoints principales	10
10.Modelo de datos	11
10.1. usuarios_db	12
10.2. canchas_db	12
10.3. reservas_db	12
10.4. reportes_db	12
11.Infraestructura y despliegue local	13
12.Seguridad e integracion	13
13.Pruebas y validacion	13

14.Entregables	15
15.Rubrica de evaluacion	16
15.1. Niveles de logro	16
16.Criterios de aceptacion del alcance	17
17.Estado actual y pendientes	17
18.Conclusiones	17
19.Anexos	18
19.1. Equipo	18
19.2. URLs locales	18
19.3. Repositorio	18

1. Resumen ejecutivo

El proyecto implementa un sistema web para reserva de canchas deportivas de padel, tenis y basquet. La solución permite a usuarios finales registrarse, iniciar sesión, consultar disponibilidad, crear reservas, revisar su historial y cancelar reservas. También permite a administradores gestionar canchas, usuarios, reservas globales, bloqueos de mantenimiento y reportes básicos de ocupación.

La arquitectura se construye con microfrontends Angular integrados mediante Module Federation, microservicios Spring Boot y una base PostgreSQL independiente por servicio. El sistema completo puede levantarse localmente con Docker Compose y expone documentación Swagger/OpenAPI en los cuatro microservicios. La validación funcional manual fue realizada; quedan pendientes las pruebas automatizadas end-to-end y la consolidación formal de evidencias de prueba.

2. Introduccion

El proyecto corresponde al trabajo integrador de la asignatura Desarrollo de Aplicaciones Empresariales. Su propósito es aplicar conceptos de arquitectura empresarial moderna mediante una solución acotada, verificable y desplegable localmente. El dominio elegido, reserva de canchas deportivas, permite trabajar reglas de negocio relevantes como disponibilidad, no solapamiento de horarios, roles diferenciados, estados de reserva y reportes operativos.

El alcance técnico prioriza independencia entre componentes. En frontend se separan responsabilidades en un shell y tres remotes. En backend se dividen dominios en cuatro microservicios. En datos se mantiene la propiedad por servicio, evitando claves foráneas y consultas cruzadas entre bases de microservicios distintos.

3. Objetivos

3.1. Objetivo general

Desarrollar un sistema web de reserva de canchas deportivas que permita a usuarios finales y administradores operar los flujos principales del dominio, utilizando microfrontends con Module Federation, microservicios Spring Boot y persistencia PostgreSQL.

3.2. Objetivos específicos

- Implementar un shell Angular que orqueste microfrontends remotos.
- Construir microservicios independientes para usuarios, canchas, reservas y reportes.
- Modelar una base PostgreSQL independiente por microservicio.

- Implementar autentificacion JWT y control de acceso por roles.
- Aplicar reglas de negocio de reserva, disponibilidad y cancelacion.
- Proveer despliegue local reproducible mediante Docker Compose.
- Documentar arquitectura, modelo de datos, APIs, pruebas y pendientes.

4. Alcance funcional

El sistema cubre los siguientes procesos principales:

- Registro e inicio de sesion de usuarios.
- Consulta de canchas activas y horarios disponibles.
- Creacion de reservas por parte de usuarios finales.
- Consulta y cancelacion de reservas propias.
- Gestion administrativa de canchas, horarios, estados y bloqueos.
- Gestion administrativa de usuarios.
- Consulta administrativa de reservas globales.
- Reportes basicos de ocupacion, demanda y cancelaciones.

Quedan fuera de alcance pagos, notificaciones automaticas, reservas recurrentes, torneos, aplicacion movil nativa y reportes avanzados de inteligencia de negocio.

4.1. Detalle de funcionalidades clave

Funcionalidad	Cumplimiento en el sistema
Consulta de disponibilidad	Permite a usuarios autenticados seleccionar cancha y fecha para visualizar bloques horarios disponibles y ocupados dentro del horario de atencion configurado.
Reserva de cancha	Permite al usuario final seleccionar deporte, cancha, fecha y bloque horario disponible. Antes de confirmar, el sistema valida disponibilidad y evita solapamientos.
Cancelacion de reserva	El usuario final puede cancelar reservas propias no iniciadas. El administrador puede cancelar cualquier reserva del sistema desde el modulo administrativo.
Gestion de canchas	El administrador puede crear, editar e inactivar canchas, definiendo nombre, deporte, horario de atencion y estado operativo.

Gestion de horarios y bloqueos	El administrador puede registrar bloqueos de mantenimiento que impiden nuevas reservas durante el intervalo configurado.
Reportes basicos	El administrador puede consultar indicadores de ocupacion, reservas por periodo, cancelaciones y demanda por cancha o deporte.

5. Roles, permisos y modulos

5.1. Roles

Rol	Permisos principales
Usuario final	Registrarse, iniciar sesion, consultar disponibilidad, crear reservas, consultar sus reservas y cancelar reservas propias antes del inicio.
Administrador	Gestionar canchas, bloqueos, usuarios, reservas globales y reportes. Puede cancelar cualquier reserva del sistema.

5.2. Modulos funcionales

Modulo	Componente	Responsabilidad
Shell	frontend/shell	Layout, autenticacion, navegacion, guards y carga de remotes.
Cliente	mf-clientes	Disponibilidad, reserva y consulta de reservas propias.
Administracion	mf-administracion	Canchas, usuarios y reservas administrativas.
Reportes	mf-reportes	Consulta de indicadores basicos para administradores.

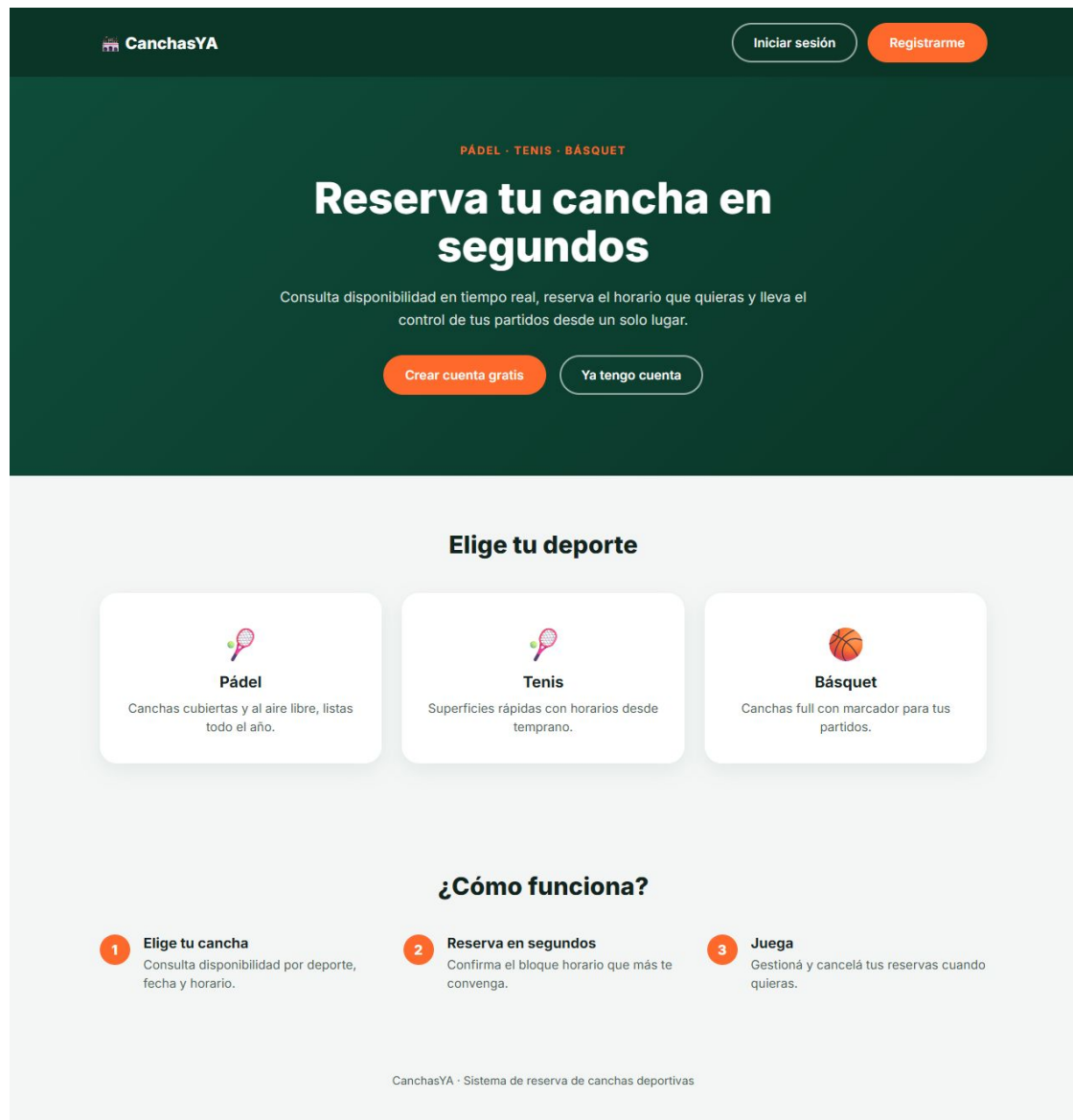


Figura 1: Pantalla inicial del shell del sistema.

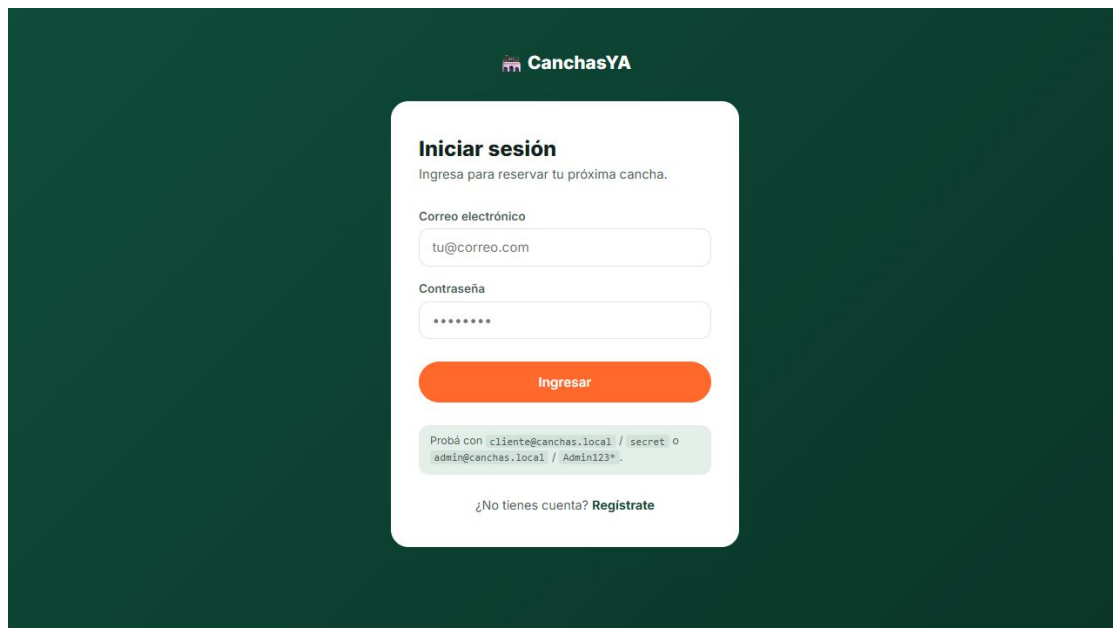


Figura 2: Pantalla de inicio de sesión.

6. Reglas de negocio

Las reglas de negocio implementadas se concentran en el proceso de reserva y en la administración de canchas:

RN	Descripción y cumplimiento
RN-01	Una reserva se realiza sobre una cancha específica de padel, tenis o basquet, en una fecha y bloque horario definido.
RN-02	No se permite crear una reserva sobre un bloque ocupado para la misma cancha. La validación se aplica en backend y se refuerza en PostgreSQL.
RN-03	El usuario final solo puede cancelar sus reservas propias; el administrador puede cancelar cualquier reserva.
RN-04	Una reserva solo puede cancelarse si su hora de inicio aun no ha ocurrido.
RN-05	Una reserva cancelada libera el bloque horario para nuevas reservas.
RN-06	El sistema contempla un límite configurable de reservas activas simultáneas por usuario.
RN-07	Solo el administrador puede crear, editar o inactivar canchas y definir su horario de atención.
RN-08	Toda reserva mantiene trazabilidad mediante los estados CONFIRMADA, CANCELADA o FINALIZADA.

La regla de no solapamiento se refuerza en PostgreSQL mediante un constraint **EXCLUDE USING gist** sobre la tabla **reservas**. Esto evita condiciones de carrera que una validación solamente en Java no podría prevenir de forma completa.

7. Arquitectura implementada

La arquitectura implementada combina microfrontends, microservicios y bases de datos independientes. El navegador carga el shell en el puerto 4300; este shell integra remotes en los puertos 4201, 4202 y 4203. Los microfrontends consumen directamente las APIs publicas de los microservicios. No se utiliza API Gateway ni BFF.

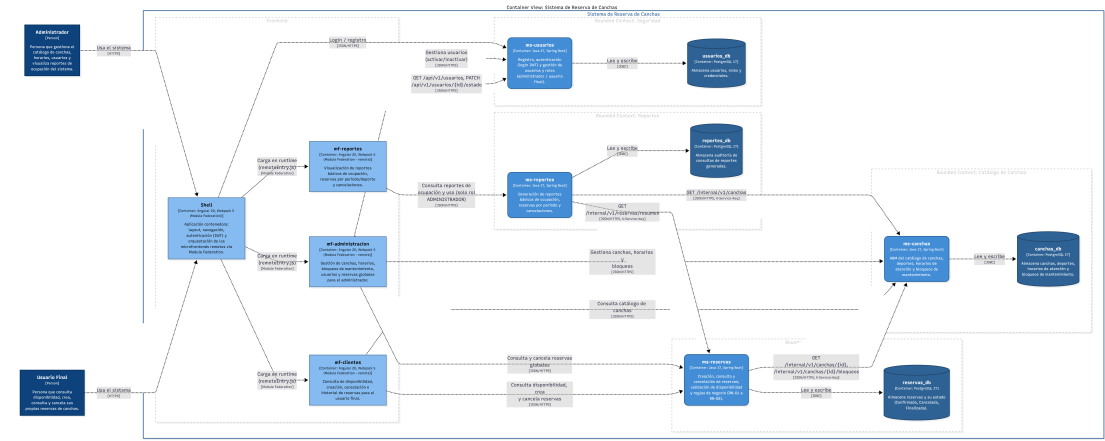


Figura 3: Vista de contenedores del sistema.

Capa	Tecnologia	Detalle
Frontend	Angular 20, Module Federation	Shell y tres remotes.
Backend	Java 17, Spring Boot, Maven	Cuatro microservicios independientes.
Datos	PostgreSQL, Flyway	Una base de datos por servicio.
Despliegue local	Docker Compose	Levanta frontend, backend y bases de datos.
Documentacion API	Swagger/OpenAPI	Disponible en runtime por micro-servicio.

8. Microfrontends

El frontend se estructura en cuatro aplicaciones Angular:

- **shell**: host de Module Federation, responsable del layout, autenticacion, rutas protegidas y navegacion.
- **mf-clientes**: remote de usuario final, enfocado en disponibilidad y reservas.
- **mf-administracion**: remote administrativo para canchas, usuarios y reservas.
- **mf-reportes**: remote administrativo para reportes.

Cada remote expone sus rutas mediante `./Routes`. El shell decide que remote cargar segun la ruta y el rol del usuario. Esta decision reduce acoplamiento entre modulos y permite construir y desplegar cada microfrontend de forma independiente.

Nueva reserva
Elige el día, la cancha y el horario para tu próximo partido. Mis reservas (1)

1. Elige el día de tu reserva

Sáb 22 Dom 23 Lun 24 Mar 25 Mié 26 Jue 27 Vie 28

2. Selecciona una cancha

Básquet Norte Pádel A Pádel B Tenis Central

3. Selecciona un horario

08:00 09:00 10:00 11:00 12:00 13:00 14:00 15:00
16:00 17:00 18:00 19:00

¡Reserva YA!

Figura 4: Modulo de cliente para disponibilidad y reserva.

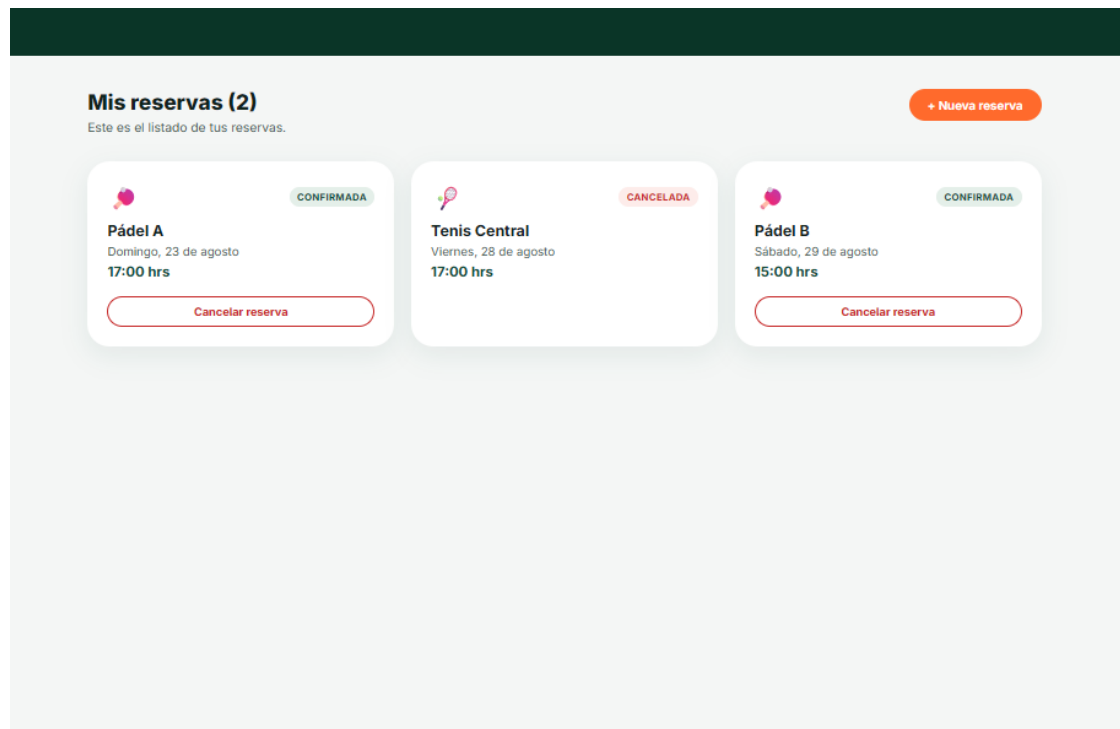


Figura 5: Modulo de cliente para consulta de reservas propias.

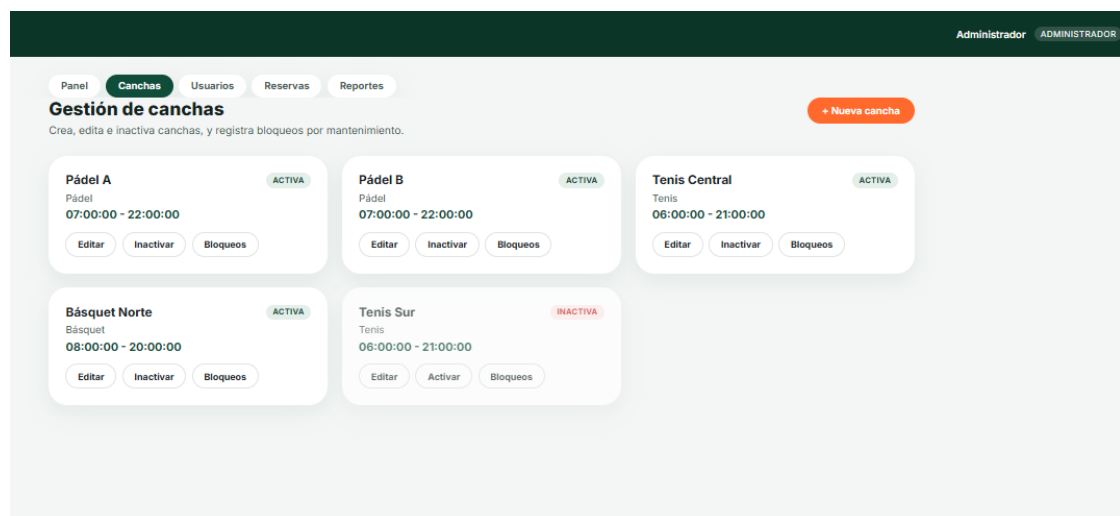


Figura 6: Modulo administrativo de canchas.

Administrador ADMINISTRADOR				
Panel	Canchas	Usuarios	Reservas	Reportes
Gestión de usuarios				
Activa o inactiva el acceso de clientes y administradores al sistema.				
NOMBRE	CORREO	ROL	ESTADO	
Carlos Mendoza	carlos@example.com	CLIENTE	ACTIVO	Inactivar
Maria Torres	maria@example.com	CLIENTE	ACTIVO	Inactivar
Juan Pérez	juan@example.com	CLIENTE	ACTIVO	Inactivar
Ana Garcia	ana@example.com	CLIENTE	ACTIVO	Inactivar
Luis Ramos	luis@example.com	CLIENTE	INACTIVO	Activar
Sofia Vega	sofia@example.com	ADMINISTRADOR	ACTIVO	Inactivar
Cliente Demo	cliente@canchas.local	CLIENTE	ACTIVO	Inactivar
Administrador	admin@canchas.local	ADMINISTRADOR	ACTIVO	Inactivar

Figura 7: Modulo administrativo de usuarios.

Administrador ADMINISTRADOR				
Panel	Canchas	Usuarios	Reservas	Reportes
Reservas del sistema				
Listado global de reservas. Como administrador puedes cancelar cualquiera.				
 Pádel A Carlos Mendoza 2026-08-22 · 09:00 hrs FINALIZADA	 Tenis Central Maria Torres 2026-08-23 · 10:00 hrs FINALIZADA	 Pádel A Cliente Demo 2026-08-23 · 17:00 hrs CONFIRMADA Cancelar reserva		
 Pádel B Juan Pérez 2026-08-24 · 08:00 hrs FINALIZADA	 Pádel B Juan Pérez 2026-08-24 · 15:00 hrs CANCELADA	 Básquet Norte Ana Garcia 2026-08-25 · 14:00 hrs FINALIZADA		
 Tenis Central Carlos Mendoza 2026-08-26 · 16:00 hrs FINALIZADA	 Pádel A Maria Torres 2026-08-27 · 11:00 hrs CANCELADA	 Tenis Central Cliente Demo 2026-08-28 · 17:00 hrs CANCELADA		

Figura 8: Modulo administrativo de reservas.

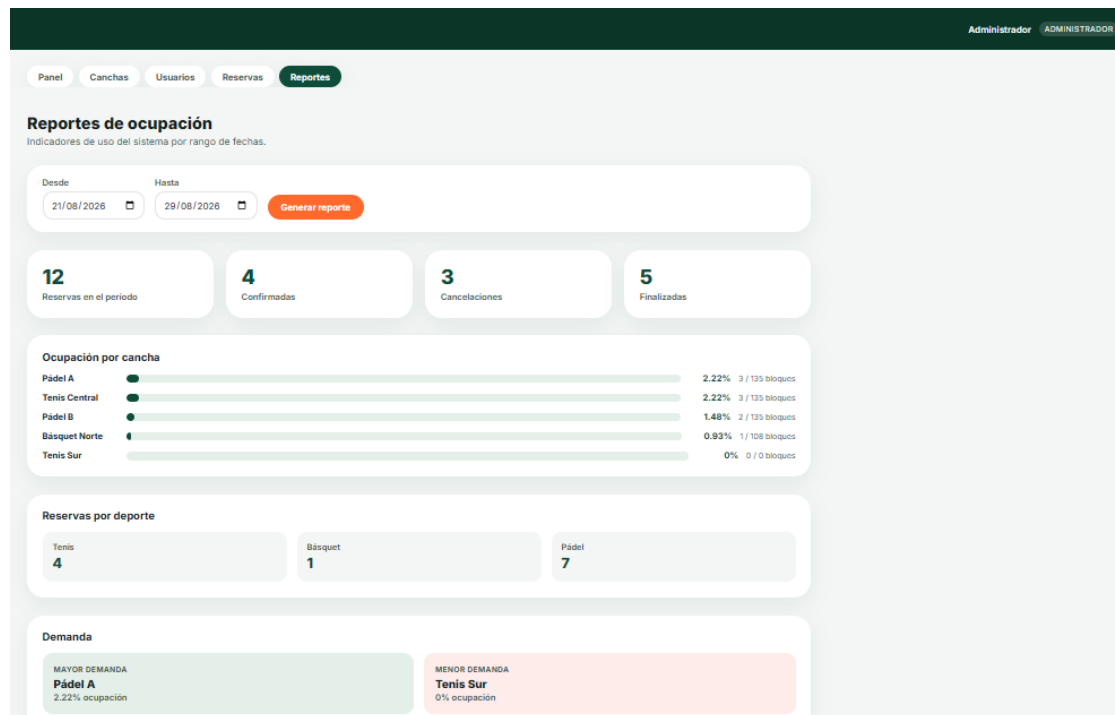


Figura 9: Modulo administrativo de reportes.

9. Microservicios

El backend esta compuesto por cuatro microservicios Spring Boot:

Servicio	Puerto	Base	Responsabilidad
ms-usuarios	8081	usuarios_db	Registro, login JWT, usuarios y roles.
ms-canchas	8082	canchas_db	Canchas, horarios, deportes y mantenimientos.
ms-reservas	8083	reservas_db	Disponibilidad, reserva, cancelacion y estados.
ms-reportes	8084	reportes_db	Reportes basicos y auditoria de consultas.

9.1. Endpoints principales

Servicio	Endpoints publicos principales
ms-usuarios	POST /api/v1/auth/registro, POST /api/v1/auth/login, GET /api/v1/usuarios/me, GET /api/v1/usuarios, PATCH /api/v1/usuarios/{id}/estado.

ms-canchas	GET /api/v1/canchas, POST /api/v1/canchas, PUT /api/v1/canchas/{id}, PATCH /api/v1/canchas/{id}/estado, POST /api/v1/canchas/{id}/mantenimientos.
ms-reservas	GET /api/v1/disponibilidad, POST /api/v1/reservas, GET /api/v1/reservas/mias, GET /api/v1/reservas, PATCH /api/v1/reservas/{id}/cancelacion.
ms-reportes	GET /api/v1/reportes/ocupacion.

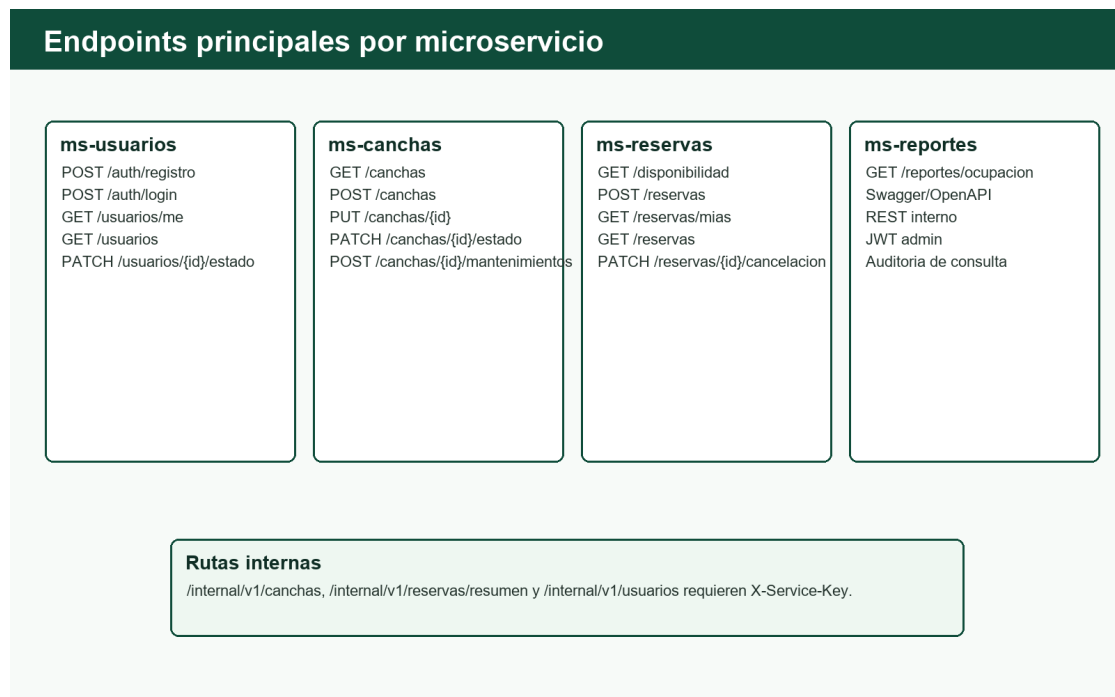


Figura 10: Resumen visual de endpoints principales.

10. Modelo de datos

Cada microservicio posee su propia base PostgreSQL. No existen joins, claves foraneas ni consultas entre bases de microservicios distintos. Las relaciones externas se guardan como UUID, por ejemplo `usuario_id` y `cancha_id` en `reservas_db`.



Figura 11: Modelo de datos por servicio.

10.1. usuarios_db

```
usuarios(id, nombre, email, password_hash, rol, activo, creado_en,
         actualizado_en)
```

10.2. canchas_db

```
canchas(id, nombre, deporte, hora_apertura, hora_cierre, activa,
        creado_en, actualizado_en)
bloqueos_mantenimiento(id, cancha_id, inicio, fin, motivo,
                       creado_en)
```

10.3. reservas_db

```
reservas(id, usuario_id, cancha_id, cancha_nombre, deporte, inicio,
         fin, estado,
         cancelada_por, motivo_cancelacion, creado_en,
         actualizado_en)
```

10.4. reportes_db

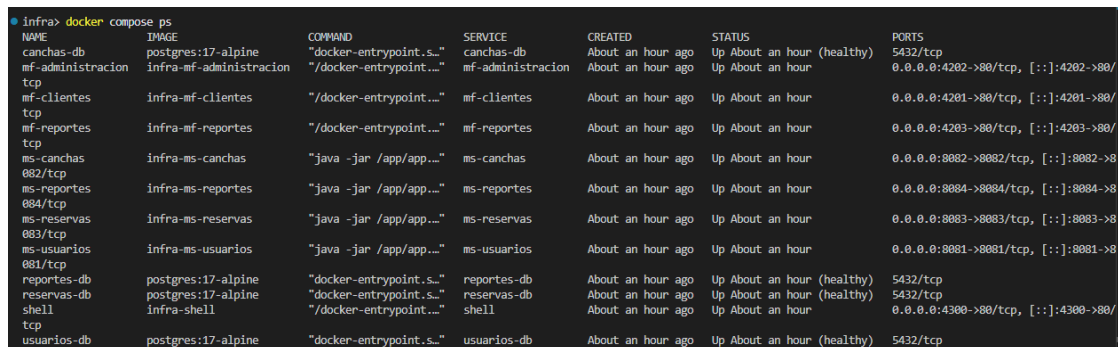
```
reporte_consultas(id, usuario_id, fecha_desde, fecha_hasta,
total_reservas, generado_en)
```

11. Infraestructura y despliegue local

El despliegue local se realiza con `infra/docker-compose.yml`. El archivo levanta cuatro contenedores PostgreSQL, cuatro microservicios, los tres microfrontends remotos y el shell.

```
cp infra/.env.example infra/.env
docker compose -f infra/docker-compose.yml up --build
```

La aplicacion se utiliza desde <http://localhost:4300>. Los puertos 4201, 4202 y 4203 corresponden a los remotes y se usan principalmente para inspeccion aislada.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
canchas-db	postgres:17-alpine	"docker-entrypoint.s..."	canchas-db	About an hour ago	Up About an hour (healthy)	5432/tcp
mf-administracion	infra-mf-administracion	"/docker-entrypoint..."	mf-administracion	About an hour ago	Up About an hour	0.0.0.0:4202->80/tcp, [::]:4202->80/tcp
mf-clientes	infra-mf-clientes	"/docker-entrypoint..."	mf-clientes	About an hour ago	Up About an hour	0.0.0.0:4201->80/tcp, [::]:4201->80/tcp
mf-reportes	infra-mf-reportes	"/docker-entrypoint..."	mf-reportes	About an hour ago	Up About an hour	0.0.0.0:4203->80/tcp, [::]:4203->80/tcp
ms-canchas	infra-ms-canchas	"java -jar /app/app..."	ms-canchas	About an hour ago	Up About an hour	0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
ms-reportes	infra-ms-reportes	"java -jar /app/app..."	ms-reportes	About an hour ago	Up About an hour	0.0.0.0:8084->8084/tcp, [::]:8084->8084/tcp
ms-reservas	infra-ms-reservas	"java -jar /app/app..."	ms-reservas	About an hour ago	Up About an hour	0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp
ms-usuarios	infra-ms-usuarios	"java -jar /app/app..."	ms-usuarios	About an hour ago	Up About an hour	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
reportes-db	postgres:17-alpine	"docker-entrypoint.s..."	reportes-db	About an hour ago	Up About an hour (healthy)	5432/tcp
reservas-db	postgres:17-alpine	"docker-entrypoint.s..."	reservas-db	About an hour ago	Up About an hour (healthy)	5432/tcp
shell	infra-shell	"/docker-entrypoint..."	shell	About an hour ago	Up About an hour	0.0.0.0:4300->80/tcp, [::]:4300->80/tcp
usuarios-db	postgres:17-alpine	"docker-entrypoint.s..."	usuarios-db	About an hour ago	Up About an hour (healthy)	5432/tcp

Figura 12: Evidencia de despliegue local con Docker Compose.

12. Seguridad e integracion

La autenticacion de usuarios se realiza con JWT emitido por `ms-usuarios`. El shell almacena la sesion localmente y usa guards para controlar la navegacion por rol. La autorizacion efectiva se valida en backend mediante configuracion de seguridad Spring.

La comunicacion interna entre microservicios usa endpoints bajo `/internal/**` protegidos con la cabecera `X-Service-Key`. Esta llave separa el trafico interno del trafico de usuarios finales.

Las credenciales incluidas en archivos de ejemplo son solo para demostracion local. En ambientes reales deben reemplazarse por variables de entorno no versionadas.

13. Pruebas y validacion

El proyecto incluye pruebas unitarias iniciales en backend y frontend, Swagger/OpenAPI disponible en runtime y una coleccion Postman con endpoints y flujo funcional completo.

Tipo	Evidencia	Estado
Unitarias backend	Tests backend/ms-*/src/test	en Implementadas inicialmente.
Unitarias frontend	Specs Angular en shell y remotes	Implementadas inicialmente.
API manual Swagger/OpenAPI	Coleccion Postman /swagger-ui.html en cada servicio	Versionada. Disponible en runtime.
Funcional manual	Flujos principales levantados con Docker Compose	Validado manualmente.
End-to-end automatizado	Cypress, Playwright u otra herramienta	Pendiente.

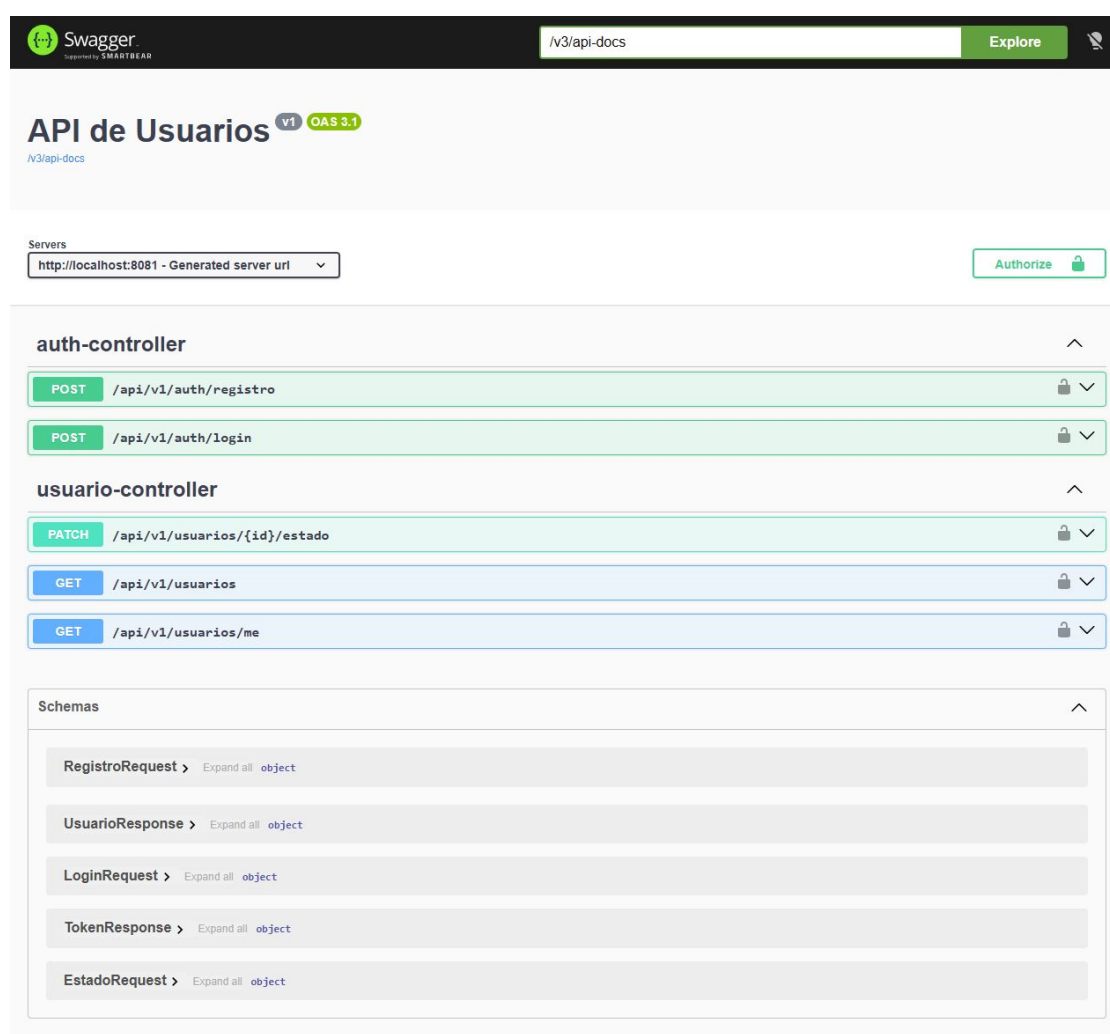


Figura 13: Swagger UI de ms-usuarios.

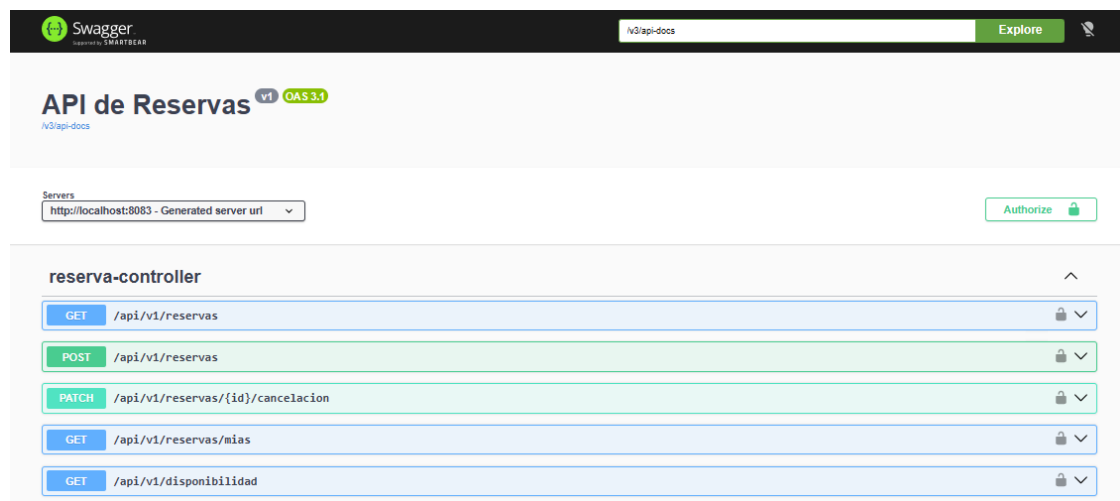


Figura 14: Swagger UI de ms-reservas.

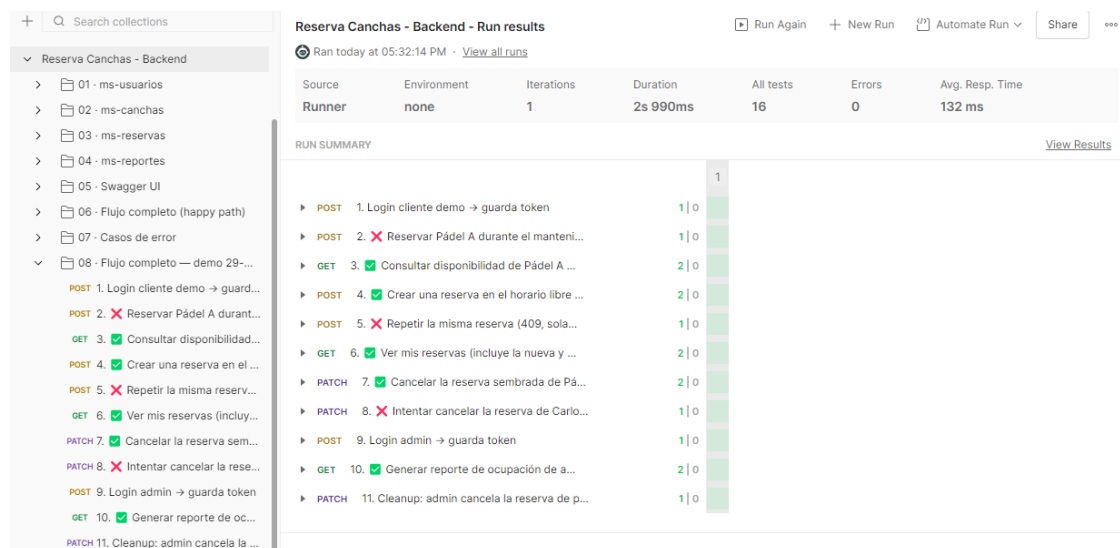


Figura 15: Coleccion Postman del backend.

14. Entregables

Codigo	Descripcion	Estado
E1	Documento de arquitectura con microfrontends, microservicios y modelo de datos.	En preparacion en LaTeX.
E2	Shell y microfrontends funcionando integrados por Module Federation.	Implementado.
E3	Microservicios Spring Boot documentados con Swagger/OpenAPI y coleccion de pruebas.	Implementado.
E4	Scripts DDL y seed de PostgreSQL.	Implementado.
E5	Manual breve de despliegue local.	En preparacion en LaTeX.

E6	Presentacion final y demostracion en vivo.	Pendiente.
----	--	------------

15. Rubrica de evaluacion

La siguiente tabla resume los criterios del alcance y el peso usado para evaluar el proyecto integrador:

Criterio	Descripcion	Peso
Alcance funcional implementado	Disponibilidad, creacion y cancelacion de reservas operativas para usuario final y administrador.	25 %
Arquitectura de microfrontends	Implementacion de Module Federation con shell host y remotes integrados, despleables de forma independiente.	15 %
Arquitectura de microservicios	Microservicios Spring Boot independientes, con responsabilidades delimitadas y API REST documentada.	15 %
Modelo de datos y persistencia	Modelo PostgreSQL con independencia de datos entre microservicios.	15 %
Reglas de negocio y validaciones	Aplicacion de RN-01 a RN-08, especialmente la validacion de solapamiento.	10 %
Modulo de reportes basicos	Reportes de ocupacion, reservas por periodo y cancelaciones con datos consistentes.	10 %
Calidad tecnica y documentacion	Buenas practicas, manual de despliegue y claridad en arquitectura y funcionamiento.	10 %
Total		100 %

15.1. Niveles de logro

Nivel	Puntaje	Descripcion general
Excelente	90–100	Cumple la totalidad de los criterios con alta calidad tecnica, reglas de negocio correctas y documentacion completa.
Bueno	75–89	Cumple la mayoria de los criterios, con observaciones menores en arquitectura, reglas o documentacion.
Suficiente	60–74	Cumple el alcance funcional minimo, pero presenta debilidades relevantes en arquitectura, datos o reglas de negocio.
Insuficiente	Menor a 60	No cumple el alcance funcional minimo o no aplica microfrontends con Module Federation y/o microservicios Spring Boot.

16. Criterios de aceptacion del alcance

El informe evidencia el cumplimiento del alcance cuando se satisfacen los siguientes criterios:

1. El usuario final puede consultar disponibilidad, crear y cancelar una reserva de canchas de padel, tenis o basquet.
2. El administrador puede gestionar el catalogo de canchas y cancelar cualquier reserva del sistema.
3. El sistema impide crear reservas sobre bloques horarios ya ocupados.
4. El modulo de reportes basicos muestra al menos ocupacion por cancha y numero de reservas por periodo.
5. El frontend esta compuesto por un shell y microfrontends remotos integrados mediante Module Federation.
6. El backend esta compuesto por microservicios Spring Boot independientes con persistencia PostgreSQL.

17. Estado actual y pendientes

El sistema esta implementado, integrado y levantado correctamente en entorno local. La validacion funcional manual esta realizada. Los pendientes principales son:

- Crear y ejecutar pruebas automatizadas end-to-end.
- Consolidar evidencias formales de pruebas.
- Exportar o adjuntar contratos OpenAPI si se requiere entrega estatica.
- Preparar presentacion final y guion de demostracion.
- Renderizar los documentos PDF desde estas fuentes LaTeX.

18. Conclusiones

El proyecto cumple el objetivo principal de implementar una aplicacion empresarial distribuida con separacion clara entre frontend, backend y datos. La division en microfrontends permite aislar flujos de cliente, administracion y reportes, mientras que los microservicios separan los dominios de usuarios, canchas, reservas y reportes.

La independencia de datos se respeto mediante una base PostgreSQL por servicio. La regla critica de no solapamiento de reservas fue reforzada en base de datos con un constraint especializado, lo cual mejora la consistencia del sistema bajo concurrencia. La decision de no utilizar API Gateway mantiene la arquitectura simple y suficiente para el alcance academico definido.

19. Anexos

19.1. Equipo

Rol	Responsabilidad primaria	Integrante
Frontend 1	Shell y autenticacion en interfaz; apoyo a mf-clientes.	Enzo Aliatis
Frontend 2	mf-administracion y mf-reportes.	Jordan Murillo
Backend 1	ms-usuarios y ms-canchas.	Galo Suarez
Backend 2	ms-reservas y ms-reportes.	Mauro Cadme

19.2. URLs locales

Componente	URL
Shell	http://localhost:4300
mf-clientes	http://localhost:4201
mf-administracion	http://localhost:4202
mf-reportes	http://localhost:4203
Swagger ms-usuarios	http://localhost:8081/swagger-ui.html
Swagger ms-canchas	http://localhost:8082/swagger-ui.html
Swagger ms-reservas	http://localhost:8083/swagger-ui.html
Swagger ms-reportes	http://localhost:8084/swagger-ui.html

19.3. Repositorio

El código fuente se mantiene en un monorepo público con carpetas principales `frontend/`, `backend/`, `database/`, `infra/` y `docs/`.